

[On This Page](#) >

Introduction to Backups

This guide explains how to protect your data using Clan's integrated backup system. You'll learn how to define what to back up, configure backup destinations, automate the process, and restore data when needed. Clan uses [BorgBackup](#) under the hood for secure, deduplicated, and encrypted backups.

Basic File and Directory Backups

At its simplest, a state defines which folders to back up. This configuration goes in your machine's NixOS configuration.

Example: Backing Up Application Data

```
1 {
2   clan.core.state.nextcloud = {
3     folders = [
4       "/var/lib/nextcloud" # Main application data
5       "/etc/nextcloud"    # Configuration files
6     ];
7   };
8 }
```

Key points:

- Specify absolute paths to directories you want to protect
- Multiple folders can be listed per state
- The backup system will automatically include all files and subdirectories

Hooks

Hooks allow you to execute custom scripts during the backup lifecycle. Use them to prepare data, stop services, or perform cleanup.

Available Hook Types

Hook	When It Executes	Common Use Cases
<code>preBackupScript</code>	Before backup starts	Stop services, dump databases, sync files
<code>postBackupScript</code>	After backup completes	Restart services, cleanup temp files
<code>preRestoreScript</code>	Before restoration starts	Prepare system, stop conflicting services
<code>postRestoreScript</code>	After restoration completes	Restart services, verify integrity

Example: Pre and Post Backup Scripts

In the following example, we configure the `nextcloud` service to:

1. Stop the relevant services before syncing its data.
2. Start the services back up after syncing its data.

```
1 clan.core.state.nextcloud = {
2   folders = [ "/var/lib/nextcloud" ];
3   preBackupScript = ''
4     export PATH=${
5       lib.makeBinPath [
6         config.systemd.package
7       ]
8     }
9
10    systemctl stop phpfpd-nextcloud.service
11    systemctl stop nextcloud-cron.timer
12  '';
13
14  postBackupScript = ''
```

```
15     export PATH=${
16         lib.makeBinPath [
17             config.systemd.package
18         ]
19     }
20
21     systemctl start phpfpd-nextcloud.service
22     systemctl start nextcloud-cron.timer
23 };
24 };
```

♥ Example: Pre And Post Restore Scripts

>

Database Backups

The manual approach above works, but Clan provides built-in support for PostgreSQL databases with proper integration.

PostgreSQL Backup Integration

Enable PostgreSQL backups and define databases to protect:

```
1 {
2     # Enable the PostgreSQL backup module
3     clan.core.postgresql.enable = true;
4
5     # Configure each database
6     clan.core.postgresql.databases.nextcloud = {
7         # Database creation options (runs on first setup)
8         create = {
9             TEMPLATE = "template0";
10            LC_COLLATE = "C";
11            LC_CTYPE = "C";
12            ENCODING = "UTF8";
13            OWNER = "nextcloud";
14        };
15
16        # Services to stop during restore (for consistency)
17        restore.stopOnRestore = [
```

```
18     "phpfpm-nextcloud.service"  
19     "nextcloud-cron.timer"  
20 ];  
21 };  
22 }
```

What this does:

- Automatically dumps the database before each backup
- Stores dumps securely in the backup repository
- Manages service dependencies during restore
- Recreates the database with correct settings on new deployments

← Previous

Using Age Plugins with Clan Vars

Next →

Minimal Example

Need help? Reach out to [our community](#).